WIP_CI_Documentation

Purpose: Make base documentation on confidence interval workflow so that people can use it to remake figures.

Here is a link to the code where confidence intervals were implemented and example code was pulled from in this documentation.

1. Install Development Branch of IEAnalyzeR

The confidence interval functionality is still in a development branch (feat_NoLineMissData) of IEAnalyzeR. Please excuse the inaccurate branch name.

I installed the development branch this way.

```
devtools::install_github(
  "https://github.com/Gulf-IEA/IEAnalyzeR/tree/feat_NoLineMissData", force = T)
```

2. Get Confidence Interval Values

The IEAnalyzeR package does not calculate the confidence intervals for you. You must provide the values either through calculating it, or if it was provided from you data source.

I'm using the MMTD_bycatch data as an example. The credible interval values were provided in a table. This dataframe has two subindicators (bycatch and mortality). I set it up so that the CI values were in their own columns with associated species names.

This is the dataframe that was compiled that includes the 95% credible intervals.

```
tab3_otter<-data.frame(year=rep(2016:2020, 2),
                       type=rep(c("Bycatch", "Mortality"), each=5),
                       kemps=c(113,257,228,411,393,38,85,75,136,127),
                       kemps_low95=c(19,47,38,62,75,6,15,12,19,23),
                       kemps_high95=c(646,1162,1029,2230,1853,206,395,359,809,610),
                       leatherback=c(12,23,22,20,20,4,8,7,7,7),
                       leatherback_low95=c(2,5,5,3,3,1,1,2,1,1),
                       leatherback_high95=c(54,97,79,74,79,18,33,28,25,28),
                       loggerhead=c(22,46,40,35,35,7,15,13,12,12),
                       loggerhead_low95=c(5,11,10,8,8,2,4,3,2,3),
                       loggerhead_high95=c(82,156,146,135,124,29,55,53,47,41),
                       green=c(150,156,144,133,141,50,51,48,45,47),
                       green_low95=c(57,59,52,54,54,18,19,17,16,16),
                       green_high95=c(355,352,360,301,337,127,119,127,113,117),
                       unknown=c(78,83,78,76,76,26,28,26,25,25),
                       unknown_low95=c(21,25,23,25,24,7,8,8,8,8),
                       unknown_high95=c(197,211,198,195,189,70,72,69,64,68))
```

3. Create upper and lower CI bounds tables

The function/workflow works with separate data frames for the upper and lower bounds. I created these data frames by selecting the "low95" columns for my lower bounds, and "high95" for my upper bounds data frames. However, you can create these data frames however you want. It will likely be a different process if you are calculating the values yourself.

```
#Otter Trawl CI
lowerCI<-select(tab3_otter, year, type, contains("low"))
lowerCI_wd<-pivot_wider(lowerCI, id_cols=year,
                        names_from = type, values_from = !c(year, type))

upperCI<-select(tab3_otter, year, type, contains("high"))
upperCI_wd<-pivot_wider(upperCI, id_cols=year,
                        names_from = type, values_from = !c(year, type))
```

The goal of the dataframe format is to match the format that would be used in the `convert_cleaned_data()` function.

For example, with the base data for this example. The data structure has year in the left column, then has each species values for both bycatch and morality. The separate lower and upper data frames should match this format exactly, however, instead of the values being the

core values, it will be either the upper or lower CI (depending on which df you are working on).

4. Run Convert Cleaned Data on all datasets

In our current workflow we have a section for correctly assigning indicator, unit, and extent names to the dataframe so that it has the correct headings. We apply this same system, with the same indicator, unit, and extent names to the upper and lower bounds data frames so that the data matches up later in the workflow. To do this we use the new `convert_ci_data()` function. This is essentially just a wrapper of the `convert_cleaned_data()` function that organizes the lower and upper data frames.

Example where the main data get converted

```
#Define header components for the data rows (ignore year). Fill in the blanks here.
indicator_names = c(rep(c("kemps", "leatherback", "loggerhead", "green", "unknown"),
                        each=2))
unit_names = c(rep(c("Bycatch", "Mortality"), 5))
extent_names = c(rep("Gulf of America", 10))

formatted_data_otter =
  IEAnalyzeR::convert_cleaned_data(tab3_otter_no_wide,
                                   indicator_names, unit_names, extent_names)
```

Example using the new `convert_ci_data()` function using the same naming vectors above.

```
ci_df_otter<-convert_ci_data(lowerCI_wd, upperCI_wd,
                             indicator_names, unit_names, extent_names)
```

Important Note: You should be using the same `indicator_names`, `unit_names`, and `extent_names` objects in the `convert_ci_data()` function. That is why it is important that the data structure is the same among the main dataframe, the upper bounds dataframe, and the lower bounds dataframe. The only difference should be the values in those dataframes (core values, upper bounds, and lower bounds).

5. Integrate CI df into the Data Prep function

Next up in our workflow is creating the data prep object using function `data_prep()`. The `data_prep()` function now has the `ci_df` argument. If you have a confidence interval object (created by `convert_ci_data()`) you will be able to add it into the data prep obect this way.

```
data_obj_otter<-data_prep(formatted_data_otter, trends = F,
                          subind = "unit", ci_df = ci_df_otter)
```

6. Plotting with CI

Once the confidence interval data has been incorporated into the data_prep object, it can be easily used in plotting. If the `plot_fn_obj()` detects CI it will automatically plot with CI. There is now a ci argument included in `plot_fn_obj()`. It will default to plotting with CI, if no CI data is found it will not attempt to plot it, it can also be turned off with `ci=FALSE`.

In this code example, the plotting code does not change because you do not need to alter any arguments for the confidence intervals to be plotted. If the data is found it will be included, but has the option to be turned off.

```
plot_otter<-IEAnalyzeR::plot_fn_obj(df_obj = data_obj_otter, trend = F,
                                    facet_grid = c("indicator", "unit"),
                                    manual_ylab = "Species", manual_title = NA)
```

7. Next Steps

There are currently no arguments related to customizing how confidence intervals are drawn. As you explore, please come up with changes to the default visualization you would like implemented or ideas for customizations.